

CaiNghien Focus Guard

One-page repo summary

Windows app

WHAT IT IS

A Windows desktop focus-enforcement app written in Python with PySide6. It combines scheduled website blocking, strict lock screens, study-session controls, a background Windows service, and a built-in updater.

WHO IT IS FOR

A Windows user who wants stronger guardrails against social-media or general computer distraction and prefers scheduled enforcement over passive reminders.

WHAT IT DOES

- Two protection modes: normal scheduled blocking and strict full-screen lock mode.
- Per-day weekly schedule with pre-lock warnings and manual lock countdowns.
- Custom blocked and allowed domains enforced through the Windows hosts file.
- Study sessions with profiles, allowed resources/apps, blocked apps/processes, and study schedules.
- Password-protected strict mode with PBKDF2 hashing and recovery-code flows.
- Background Windows service with startup self-heal, UI heartbeat checks, and event logging.
- In-app update checks and installer downloads from the repo's static release site.

HOW TO RUN

Windows only.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python app.py
```

Run as Administrator if you want hosts-file blocking to apply.

HOW IT WORKS

- Desktop UI: `app.py` launches the PySide6 `MainWindow`; the UI talks to `EnforcementController` for config saves, protection state, manual locks, study sessions, updates, and uninstall approval.
- Rules and enforcement: `runtime\_rules.py` decides current and upcoming protection windows. `hosts\_blocker.py` rewrites the Windows `hosts` file, and `process\_guard.py` kills blocked apps when study enforcement is active.
- Data and security: `ConfigStore` persists `config.json`, `state.json`, and `events.jsonl` under `%APPDATA%` or `%PROGRAMDATA%`, with backups, cached installers, and weekly stats. `security.py` hashes passwords and recovery codes with PBKDF2.
- Service flow: `service.py` / `service\_host.py` run a Windows service that reloads config and state, reapplies hosts blocking, applies delayed config changes, watches UI heartbeat, and repairs startup/service state.
- Release flow: `web/` is a separate static site built with Node and configured for Render. It serves installer downloads plus `web/data/update-manifest.json`, which `UpdateManager` fetches and uses to download newer installers.
- Backend/API: Not found in repo; update metadata is served as static JSON rather than a dynamic backend.